

COVID-19 DATA ANALYSIS, INTEGRATION, AND VISUALIZATION PLATFORM

Data Engineering Bootcamp

Prepared by: Deimantas Baltrėnas

2026-07-30

TABLE OF CONTENTS

INTRODUCTION	4
1. TASK 1 – SNOWFLAKE ACCOUNT SETUP AND DATASET ACQUISITION	5
1.1. Environment	5
1.2. Dataset Acquisition	5
2. TASK 2 – DATA EXPLORATION AND ENHANCEMENT	6
2.1. Identification on the Primary Source Table.....	6
2.2. Reporting Granularity.....	6
2.3. Data Quality Assessment.....	7
2.4. Data Enrichment.....	8
2.5. Automated Exploratory Data Analysis.....	9
3. TASK 3 – DATA MODELING IN NOSQL.....	10
3.1. Rationale.....	10
3.2. Schema Design	11
3.3. Implementation.....	13
4. TASK 4 – API DEVELOPMENT.....	13
4.1. Framework Choice	13
4.2. Architecture	14
4.3. Snowflake-Backed Endpoints	14
4.4. On-Fly Data Processing.....	15
4.5. MongoDB-Backed Endpoints	15
5. TASK 5 – INTERACTIVE VISUALIZATION	15
5.1. Technology Choice.....	15
5.2. Architecture	16
5.3. Visualizations	16
5.4 Bonus: Annotations from Dashboard	17
5.5 Data Handling Issues Found During Testing	17
6. TASK 6 – ANALYTICAL FEATURES.....	18
6.1 Technology Choice.....	18
6.2 Time Series Forecasting	18
6.3 Bonus: Clustering	19
7. TASK 7 – PERFORMACE OPTIMIZATION	19
7.1 Constraint Specific to Dataset	19
7.2 Bottleneck.....	20
7.3 Solution: An Owned, Pre-Aggregated Dynamic Table	20
7.4 Measured Result	21
8. TASK 8 – CACHING	22
8.1 What Gets Cached and Why.....	22
8.2 Design: A Shared In-Memory Cache, Not an External Service.....	23
8.3 Verifying Cache Actually Works	23
9. TASK 9 – PATTERN RECOGNITION	24
9.1 What Pattern to Look For	24
9.2 Constraint With PREV().....	24

9.3 Single-Day Noise Breaks Long Trends	24
9.4 Result	25
10. TASK 10 – CODE AND REPOSITORY	25
CONCLUSION AND INSIGHTS SUMMARY	27

INTRODUCTION

COVID-19 – respiratory illness cause by SARS-CoV-2 coronavirus, was first identified in Hubei Province, China, in late 2019. Within a few months it had spread to every continent except Antarctica, and the World Health Organization (WHO) declared it a pandemic on 11 March 2020. Scale and duration of an outbreak generated an unusually large volume of public epidemiological data: case counts, deaths, recoveries, vaccination progress, and government policy responses. It recorded consistently across countries and, in several cases, down to the state or country level. This combination of scale, geographic granularity, and public availability makes COVID-19 dataset a practical case study for applying modern data engineering practices, which is basis for this project.

This project is a bootcamp assignment for a data engineering bootcamp. The assignment specifies which technologies to use (Snowflake for structured storage, Python for back-end and analysis, NoSQL database for supplementary data, and Python visualization library) and a fixed set of tasks, from acquiring dataset through performance optimization and pattern recognition. Purpose is to apply what was covered during the bootcamp to a single, realistic dataset: querying and modeling data in Snowflake, augmenting it with external sources, designing a NoSQL schema for the kind of data that does not fit relational structure, building an API on top of both, and visualizing a result.

Platform combines structured and semi-structured sources, exposes resulting data through an API, and presents it through interactive visualizations. Intent is to turn a large public dataset into something that supports analysis of how pandemic spread and how it affected different regions and populations.

Reasoning behind each technical decision is documented alongside its implementation throughout this report. That matters because choices made early in the project, such as: how the source data is structured, how it behaves when reported, and where its limits are, shape what is possible in everything built afterward.

Repository: <https://github.com/DeimantasBaltrenas/covid19-project>

1. TASK 1 – SNOWFLAKE ACCOUNT SETUP AND DATASET ACQUISITION

1.1. Environment

A Snowflake trial account was provisioned on AWS in Stockholm region (*eu-north-1*), as specified in project requirements. Account identifier is *XENKFUW-JJ81226*, accessible at *ib94311.eu-north-1.aws.snowflakecomputing.com*.

1.2. Dataset Acquisition

Dataset selected is COVID-19 Epidemiological Data, published by Starschema on the Snowflake Marketplace as a Secure Data Share. Secure Data Sharing consumes the provider's data without copying it into the consumer's account. The dataset is accessible as an *IMPORTED DATABASE* object, *COVID19_EPIDEMIOLOGICAL_DATA*, and this has two consequences that matter for the rest of the project. No storage cost is incurred on the consumer side, it's relevant, since accounts operate under a trial credit allowance and a resource monitor intended to conserve that allowance. Second, updates to the dataset are propagated by the provider automatically. Platform therefore does not need to implement or schedule its own ingestion process for this data source, which freed development effort for analysis and enrichment work.

The database consists of 43 tables within the *PUBLIC* schema. This cover confirmed case counts, deaths, recoveries, vaccination progress, hospital capacity indicators, government policy response data, and demographic reference tables. All tables in this share were found to be fully structured, with no semi-structured column types (*VARIANT*, *OBJECT*, or *ARRAY*). This finding directly informed a design decision for the platform. The project brief requires the combination of structured and semi-structured data sources, and since the Snowflake side of the platform does not itself contain semi-structured data, that requirement is instead satisfied by the MongoDB component developed in Task 3 — which stores annotations and external references as documents with variable structure. The two halves of the platform were therefore not designed independently. The absence of semi-structured data on the Snowflake side is the reason a NoSQL layer is necessary at all, not an optional addition.

2. TASK 2 – DATA EXPLORATION AND ENHANCEMENT

2.1. Identification on the Primary Source Table

Dataset contains three tables with similar naming and purpose. Using wrong one as a foundation would have meant every subsequent query in this report was built on incorrect premises. For that reason, three were compared directly, rather than one being selected by assumption:

Table	Row Count
JHU_COVID_19	9,738,292
JHU_COVID_19_TIMESERIES	12,450,699
JHU_DASHBOARD_COVID_19_GLOBAL	4,632

A reasonable first guess, going only by names, would be that *JHU_COVID19_TIMESERIES* is a reformatted duplicate of *JHU_COVID_19* and that *JHU_DASHBOARD_COVID19_GLOBAL* is just a smaller time series meant for dashboard use. Row counts rule out both guesses. *JHU_COVID19_TIMESERIES* has more rows than *JHU_COVID_19*, which points to finer granularity rather than duplication. A duplicate would be expected to match original row count, now exceed it. *JHU_DASHBOARD_COVID19_GLOBAL* has far fewer rows than either of other two, which is explained by it holding a single reporting snapshot per location rather than a history. Despite name, it is not a time series at all, and using it for trend analysis would give one data point per location instead of trend line. While speaking about dataset spans, it's range is between 2020-01-22 to 2023-03-09 and includes 216 distinct countries and regions. Both numbers were confirmed against *JHU_COVID_19* directly (row-count query, extended with *MIN(DATE)*, *MAX(DATE)*, and *COUNT(DISTINCT COUNTRY_REGION)*)

2.2. Reporting Granularity

Looking closer at source table showed that reporting granularity is not the same for every country. It decides whether a plain *GROUP BY COUNTRY_REGION, DATE* query gives a correct national total or silently wrong one. A query grouped only by *COUNTRY_REGION* and *DATE* ends up returning one row per country for every date, instead of one row per country. Any later step that treats these country-level rows as if they were already national totals, taking their maximum rather than summing them, for example, produces a number far below real national figure, and nothing about query itself signals that anything went wrong. That happened

in growth calculation described further down in this section. Fixing it depends entirely on recognizing this structural property first.

Germany has been chosen particularly because it shows that this property is not even fixed within one country over time. Records with *PROVINCE_STATE IS NULL* for Germany only exist between 2020-01-22 and 2020-05-13, after which reporting moved to the state (Bundesland) level. A query written and tested against early-2020 German data, where the one-row-per-country assumption happens to hold, would look correct during testing, but would undercount Germany from mid-May 2020 onwards, once underlying reporting structure changed underneath it. That covers checking reporting structure across a dataset's whole data range rather than for one representative period, since there is no guarantee data behaves same way throughout.

An exact problem showed up in an early version of a month-over-month growth calculation that computed *MAX(CASES)* per country per month. For a country reporting below national level, that expression returns case count of its single largest subregion for that month, not national total. Result understated cumulative US cases by roughly one to three orders of magnitude, depending on month checked. Mistake surfaced by comparing output against externally known approximate totals and noting discrepancy was too large to be plausible. Fix took two steps: summing case counts with *SUM(CASES)* across all subregions per country per day, then taking monthly maximum of that resulting daily national total. Summing first makes subregional rows be combined into a true daily national total before any further aggregation happens.

2.3. Data Quality Assessment

A duplicate-detection query grouped by *COUNTRY_REGION*, *PROVINCE_STATE*, *DATE*, *CASE_TYPE* returned 126,858 rows meeting duplicate condition, which is a large enough number to check carefully before reporting it as a data quality issue. Digging into a cause showed it is a direct consequence of a granularity problem. Multiple counties within the same state legitimately share a common province and date value, and grouping at province level without also grouping by county identifier makes every county-level row look like a duplicate of every other county in the same state on the same date, when none of them are. This distinction matters, because reporting 126,858 duplicates without identifying the cause would have badly overstated a real data quality problem. Adding *FIPS* field to grouping, so each row is identified at its true reporting unit instead of a coarser one, cut confirmed duplicate count to 8,128, a figure that reflects actual duplication rather than an artifact of grouping too loosely. Further checks covered null values in key columns, negative values in daily case figures, and gaps in

continuous daily reporting. Negative values looked like a data quality concern at first, since a negative case count has no direct physical meaning. Cross-referencing these rows against publicly known reporting practices gave a different explanation: they correspond to retroactive revisions of previously published totals, such as a jurisdiction correcting an earlier overcount by reporting a negative adjustment on a later date, rather than to data entry errors. That distinction changes how figures should be read downstream – a negative value should not be discarded as invalid, but treated as a legitimate correction affecting running cumulative total. These checks, along with case fatality rate calculations, a 7-day rolling average of new cases, and a ranking of the twenty countries by cumulative case count, are documented in full in *exploration.sql* file.

2.4. Data Enrichment

An enrichment procedure was built in Python (*automated_eda.py*) to extend Snowflake case data with external context. Case counts alone only support a limited set of questions, mostly about how the disease spread, while the assignment also asks for insight into effects and patterns across populations, which needs demographic and economic context that the case dataset does not contain on its own. Confirmed case counts are pulled with aggregation applied by country and date, summing the CASES and DIFFERENCE fields across subregions to avoid the granularity issue. This step could not be skipped, since any enrichment built on top of incorrectly aggregated case data would inherit the same undercount problem no matter how good the external data joined to it was. Vaccination coverage data comes from OWID_VACCINATIONS and is merged with the case data on country and date, extending the analysis to cover how case trends relate to vaccination progress. Two Marketplace listings were evaluated for external demographic and economic context before one was selected. The choice matters because it decides whether the enrichment step holds up for the rest of the project instead of breaking under some future usage limit. The first candidate, "World Bank: World Development Indicators," turned out to be a monetized listing priced at roughly \$80 per month, with the free tier capped at 1,000 rows, a limit that would be used up quickly given the size of the case dataset, and one that would add a recurring cost that does not fit a bootcamp project. The second candidate, "Snowflake Public Data (Free)," was confirmed to provide equivalent GDP-per-capita and population indicators, aggregated from the World Bank, the US Census Bureau, and other public sources, with no cost and no row limit. It was selected on that basis: it meets the same data requirement without adding a dependency that could fail or start costing money later in the project. The case dataset and the World Bank tables are joined using ISO alpha-2 country codes instead of country names. That choice comes from a general problem

with joining on names: two independently maintained datasets are not guaranteed to use the same string for the same country. One might write "United States," another "USA," another "US," and a join on these strings directly would silently drop every row where the two disagree on formatting, with no error to flag it. ISO codes avoid this because they are a fixed, standardized identifier rather than free text. Even so, a code-based join is not guaranteed to match every row, since the dataset can contain ISO codes for entities that do not appear as countries in the World Bank series, such as cruise ships or disputed territories with their own JHU reporting line, or the reverse. For this reason, the procedure computes and reports a match-rate diagnostic after the join, which makes unmatched records visible so they can be accounted for in the analysis instead of quietly disappearing from the merged dataset.

2.5. Automated Exploratory Data Analysis

Enriched dataset is analyzed with *ydata-profiling* library, which produces a report of variable distributions, missing-value patterns, and correlations without manual inspection of individual columns. Automating this step was a deliberate choice given the size of the enriched dataset. Checking the distribution and missingness of every column by hand across millions of rows would not be practical. An automated profiling pass catches exactly the kind of anomaly (unexpected missingness, implausible value ranges, skew) that would otherwise need to be re-checked by hand every time the underlying query changed. Reviewing this report surfaced the following characteristics of the enriched dataset, each of which is explained by a specific property of the underlying data rather than left as an unexplained observation: A clear rise in case counts around January 2022, matching the Omicron wave and confirming the trend already seen in the 7-day rolling average. Two independent views of the data, a manually computed SQL rolling average and an automatically generated profiling report, agree on the same period, which strengthens confidence that the pattern is a genuine feature of the data and not an artifact of either method. Substantial missingness in vaccination coverage fields. This is not a data quality defect, because the cause is known: OWID_VACCINATIONS only holds records from the point vaccines became available, while the case dataset starts recording in January 2020, well before that. A left join against a table with a later start date will always produce missing values for every date before that start, and that accounts for most of the observed missingness. Negative values in the derived NEW_CASES field. These come from retroactive data revisions rather than entry errors. Their presence in the enriched dataset is expected, consistent with that earlier finding, and not a new issue introduced by enrichment. A right-skewed distribution of daily case counts, consistent with the pandemic unfolding in discrete waves (periods of rapid growth followed by decline) rather than growth at a constant rate. A constant growth rate would

be expected to produce a more symmetric distribution, whereas wave-like growth concentrates a small number of very high daily values against a much larger number of comparatively low ones.

3. TASK 3 – DATA MODELING IN NOSQL

3.1. Rationale

Snowflake side of the platform is relational, which suits data with a fixed structure and defined grain, such as case counts and vaccination figures, where every record has the same set of fields and the same relationship to a specific country and date. An assignment also calls for storing supplementary content, namely annotations tied to specific data points and references to external sources, where that assumption breaks down. Number of tags on an annotation, or the number of countries referenced by an external source, is not fixed and cannot be predicted in advance.

Two approaches were considered for this content. The first was to extend the existing relational structure with more tables, such as a table of annotations with a fixed number of tag columns, or a junction table linking sources to countries. This was rejected: it requires deciding in advance on a maximum number of tags or related countries, or adding join complexity purely to handle a variable number of values that a document model represents natively as an array. The second approach, a document-oriented model, represents each annotation or source as one self-contained document with fields of variable length, and sidesteps this constraint entirely. It was adopted for a simple reason: it matches the actual shape of the data being stored, instead of forcing that data into a structure built for something else.

The NoSQL component runs on MongoDB Atlas, specifically a free-tier (M0) cluster named Covid19Cluster, provisioned on AWS in the Stockholm region. The region was matched to the Snowflake environment from Task 1 so both components sit under comparable network conditions, which is relevant for a platform that, in later tasks, queries both systems while serving a single API request.

3.2. Schema Design

Two collections were designed inside a database named *covid19_project*.

annotations stores comments tied to a specific country, date, and case type from the Snowflake dataset:

```
{
  "_id": ObjectId("..."),
  "country_region": "United States",
  "iso3166_1": "US",
  "date": ISODate("2022-01-10"),
  "case_type": "Confirmed",
  "author": "deimantasbaltrenas",
  "comment_text": "Omicron wave peak in the US, identified from the rolling 7-day average in the Task 2 analysis.",
  "tags": ["omicron", "peak", "wave"],
  "created_at": ISODate("2026-07-20T10:00:00Z"),
  "updated_at": null
}
```

The *iso3166_1* field sits alongside *country_region*: a lookup done on country names is exposed to formatting mismatches between systems, while ISO codes are not. Having this field means an annotation created against a Snowflake record can reliably be matched back to it later, even if the two systems end up spelling the country name differently, which is entirely possible.

Field	Type	Notes
<i>country_region</i>	string	Matches COUNTRY_REGION in JHU_COVID_19
<i>iso3166_1</i>	string	ISO country code, used as join key back to Snowflake
<i>date</i>	date	Matches DATE column in Snowflake table
<i>case_type</i>	string	e.g. <i>Confirmed</i> , <i>Deaths</i> , <i>Recovered</i>
<i>author</i>	string	Who wrote annotation
<i>comment_text</i>	string	Free-text note
<i>tags</i>	array of strings	Arbitrary length, used for filtering
<i>created_at</i> / <i>updated_at</i>	date/null	Audit fields

external_sources stores references to external articles, reports, or datasets tied to one or more countries and, where relevant, a date range:

```
{
  "_id": ObjectId("..."),
  "title": "Germany switches to per-state COVID reporting",
  "description": "Article describing the change in reporting granularity.",
  "url": "https://example.com/article",
  "related_countries": ["Germany"],
  "date_range": { "start": ISODate("2020-05-13"), "end": null },
  "added_by": "deimantasbaltrenas",
  "added_at": ISODate("2026-07-20T10:00:00Z")
}
```

The *date_range* field has a *start* and an *end*, and *end* is explicitly allowed to be null. This reflects that not every source describes a bounded event. An article about a permanent change in reporting methodology, like the example above, stays relevant indefinitely, not for a fixed window. A schema that forced a fixed end date onto every source would misrepresent this kind of entry.

Field	Type	Notes
title	string	Short title of source
description	string	What it's about
url	string	Link to source
related_countries	array of strings	One or more countries it relates to
date_range.start / date_range.end	date/null	<i>end:null</i> means open-ended / still relevant
added_by	string	Who added entry
added_at	date	When it was added to

The two collections are not embedded within one another. If an annotation needs to reference a specific external source, it stores that source's *_id* rather than duplicating its content inline. This follows the same reasoning that favored a document model over a relational one, just applied the other way: variable-shaped data benefits from a flexible document, but two independently useful documents should still stay independently readable, not nested inside each other in a way that makes either awkward to query or update on its own.

3.3. Implementation

The schema from Section 4.2 was implemented on the live Atlas cluster using the *pymongo* driver, in the script *setup_mongodb.py*. The script authenticates with a *MONGODB_URI* connection string read from a local *.env* file, following the same credential-handling convention set up for the Snowflake connection in Task 2. That convention was applied to both database connections in the project for a practical reason: a credential-handling approach only followed for one of two connections provides no real protection, since the project would still be unsafe to commit to a public repository if either connection had hardcoded credentials.

The collections were populated with records drawn from the findings in Section 3, not placeholder content, so the NoSQL component demonstrates its intended use rather than sitting there as an empty schema. *Anntations* collection has entries documenting the Omicron case peak, the change in Germany's reporting granularity, and the retroactive revisions found during the data quality assessment. The *external_sources* collection references the two datasets used in the enrichment work from Section 3.4: the Snowflake Public Data (Free) listing and the OWID vaccination dataset. This gives a direct, queryable link between the data sources used for analysis in Task 2 and their documentation in the NoSQL component, instead of leaving that connection recorded only in this report.

4. TASK 4 – API DEVELOPMENT

4.1. Framework Choice

API was built with FastAPI instead of Flask. Both frameworks could satisfy the assignment's requirements, but FastAPI won out for two concrete reasons. First, it validates incoming request parameters automatically against declared Python types and rejects bad input before it reaches any application code. A request to */cases* missing a *country* parameter, for instance, fails with a clear error at the framework level instead of causing an unhandled exception further down. Second, it generates an interactive API documentation page (Swagger UI) straight from the code, with no separate documentation step. That matters for a project meant to run and be tested on a machine other than the one it was built on: a supervisor evaluating the project can exercise every endpoint from a browser without writing a single line of client code.

4.2. Architecture

The API is split across three files. *api/snowflake_client.py* and *api/mongo_client.py* each expose a single connection helper, reused by every endpoint that needs Snowflake or MongoDB access. *api/main.py* holds the endpoint definitions. This split follows the same logic applied to credential handling earlier: a connection pattern written once and reused is easier to keep correct than the same logic copy-pasted into every endpoint that happens to need a database.

The Snowflake connection helper opens a new connection per request and closes it right after, using a context manager so the connection is closed even if the query fails halfway through. The MongoDB helper works differently: it creates a single client on first use and reuses it for the lifetime of the running API process. This is a deliberate difference, not an inconsistency. *pymongo's* client manages its own connection pool internally and is meant to be created once and shared, while the Snowflake connector does not offer the same pooling out of the box, so a fresh connection is opened for each request instead.

4.3. Snowflake-Backed Endpoints

Four endpoints query Snowflake directly:

- *GET /countries* returns the distinct list of countries in *JHU_COVID_19* with their ISO alpha-2 codes, meant to populate a selection control in the dashboard built later.
- *GET /cases* returns national daily case totals for a given country and case type, aggregated with *SUM(CASES)* across subregions before being returned. This is the same fix established earlier for the growth calculation: grouping case data only by country and date, without summing across subregions first, would return one row per subregion instead of one row per country for countries that report below national level. Doing this aggregation once, inside the API, means every caller of */cases* gets a correct national total without needing to know that the underlying table requires this fix at all.
- *GET /vaccinations* returns vaccination coverage by country and date from *OWID_VACCINATIONS*.
- *GET /cases/rolling-average* extends */cases* with a smoothed daily series, described below.

Each endpoint accepts optional *start_date* and *end_date* parameters, so a caller can ask for a bounded window instead of a country's entire history, which is useful given that some series run more than three years of daily records.

4.4. On-Fly Data Processing

The assignment asks the API to perform on-the-fly data processing, beyond just relaying stored values. This shows up in *GET /cases/rolling-average*, which computes a rolling average of new cases over a configurable window (7 days by default) inside the API process itself, rather than in a SQL query or a precomputed column in Snowflake. The endpoint calls the existing */cases* logic as a plain Python function, loads the result into a *pandas* DataFrame, and applies *.rolling(window=n).mean()* to the *NEW_CASES* column. Doing this in the API layer, instead of storing a pre-computed rolling average in Snowflake, means the window size becomes a request-time parameter: a caller can ask for a 7-day or a 14-day average from the same endpoint without touching the underlying data at all.

4.5. MongoDB-Backed Endpoints

Two endpoint pairs connect the API to the MongoDB component from Task 3: *GET /annotations* and *POST /annotations*, and *GET /external-sources* and *POST /external-sources*. The GET endpoints accept optional filters (by country, and for annotations also by case type) and query the corresponding collection directly. The POST endpoints accept a JSON body validated against a Pydantic model, *AnnotationCreate* or *ExternalSourceCreate*, before a new document is built and inserted. This is where the API interacts with the NoSQL database for supplementary data, and it also sets up the bonus feature in Task 5, where a dashboard user adds annotations that get stored in the NoSQL database. The write path for that feature already exists at the API layer.

All endpoints, the Snowflake-backed ones (*/countries*, */cases*, */vaccinations*) and the MongoDB-backed ones (*/annotations*, */external-sources*), were tested end to end against the live Snowflake account and the Atlas cluster from Task 3, and returned correct data through the Swagger interface.

5. TASK 5 – INTERACTIVE VISUALIZATION

5.1. Technology Choice

The dashboard was built with Dash, one of the two libraries the assignment names directly. Dash was picked over Streamlit for this project specifically because it fits naturally with the API from Task 4: a Dash application is itself a set of Python callback functions reacting

to user input, which maps cleanly onto making HTTP requests to existing API endpoints, rather than requiring a different structure to reach the same data.

5.2. Architecture

The dashboard (*dashboard/app.py*) does not talk to Snowflake or MongoDB directly. It calls the API from Task 4 over HTTP using the *requests* library and holds no database credentials of its own. This carries forward the separation of concerns set up in Task 4: the API already handles authentication, query construction, and the subregional aggregation fix described earlier, so rebuilding that logic in the dashboard would duplicate it and risk it drifting out of sync with the API's behavior over time. Running the platform means two processes at once, the API (*uvicorn api.main:app*) and the dashboard (*python dashboard/app.py*), with the dashboard treating the API exactly like any other client would.

5.3. Visualizations

Three metrics are visualized per country, covering the assignment's requirement for infection rates and mortality rates directly, with vaccination coverage substituted for a demographic breakdown, since the dataset does not include a demographic split (age bands, for example) at the country/date grain the rest of the platform operates on, while vaccination progress is available at that same grain and gives a comparable "how populations were affected differently" angle:

- **Infection rate**, new confirmed cases as a bar chart, overlaid with the 7-day rolling average from the */cases/rolling-average* endpoint built in Task 4. Plotting both together makes the smoothing effect of the rolling average visible directly against the noisier daily figures it comes from.
- **Mortality rate**, a case fatality rate (CFR), computed as deaths divided by confirmed cases and expressed as a percentage over time. This needed two separate calls to */cases*, one with *case_type=Confirmed*, one with *case_type=Deaths*, merged in the dashboard on the DATE column with *pandas.merge()*, since the API returns each case type on its own rather than one combined table. Rows with zero confirmed cases are excluded before computing the ratio, to keep a division-by-zero result out of the chart.
- **Vaccination coverage**, people fully vaccinated per hundred, from the */vaccinations* endpoint, giving a third angle to compare against the case and mortality trends for the same country and period.

A date-range control lets these three charts be viewed over the full recorded period or a shorter recent window. It was built as a fixed set of presets (last 30 days, last 90 days, last year, all time) instead of a free-form calendar picker, after the calendar picker was flagged as a usability problem during testing: it needed an extra confirmation click that a fixed preset does not. The presets are computed relative to the last date present in the dataset (2023-03-09), not the real-world current date, since the dataset does not reach the present.

5.4 Bonus: Annotations from Dashboard

Dashboard includes a form for adding an annotation to the currently selected country, plus a list of existing annotations for that country underneath the charts. Submitting the form sends a *POST* request to the */annotations* endpoint built in Task 4, which inserts the new document into the MongoDB *annotations* collection. The list shown on the page is driven by the same *GET /annotations* endpoint and refreshes automatically after a successful submission. This meets bonus requirement for this task: annotations entered through the dashboard land in the NoSQL database, using the write path already built and tested as part of Task 4, rather than a separate one built just for the dashboard.

5.5 Data Handling Issues Found During Testing

Two issues turned up while testing the dashboard against the live API, and both show why testing across the whole stack, not each piece in isolation, was necessary before calling the dashboard finished.

First, */cases/rolling-average* returned a 500 error specifically when called from the dashboard, while the underlying */cases* endpoint worked fine. The cause was a missing daily value producing a *NaN* in the computed rolling average: Starlette's default JSON response explicitly disallows *NaN* and raises an error instead of silently rendering it, while pandas' own JSON serialization converts *NaN* to null correctly. The endpoint was changed to serialize its response through *pandas.DataFrame.to_json()* instead of a plain dictionary conversion, for that reason.

Second, one annotation record had no date attached (a note about negative values in *NEW_CASES* that does not tie to a single calendar date). The dashboard's annotation list assumed a date string would always be there and threw a *TypeError* when it was not. This was fixed by treating a missing date as an empty string instead of assuming the field is always populated, a reminder that the same variable-shape reasoning behind the schema design also has to be handled by anything reading from that schema later, not only by the schema itself.

6. TASK 6 – ANALYTICAL FEATURES

6.1 Technology Choice

Forecasting was built with *statsmodels*' Holt-Winters exponential smoothing instead of a more specialized forecasting library such as Prophet. The assignment leaves the choice of library open for this task, requiring only that Python be used, and *statsmodels* was chosen for a practical reason that came up earlier in the project too: heavier, more specialized packages (*ydata-profiling* in Task 2, for example) caused real dependency and Python-version compatibility problems during development, so a library already known to be compatible with the existing environment seemed the safer pick for something that also has to run correctly on a machine other than the one it was built on. The clustering bonus used *scikit-learn*'s KMeans for the same reason: a standard, broadly compatible library rather than a specialized one.

6.2 Time Series Forecasting

The `/cases/forecast` endpoint fits a Holt-Winters model against a country's full historical daily new-case series and forecasts a configurable number of days beyond the last recorded date (14 by default). Two modeling choices follow directly from earlier findings in this report. First, the model uses an additive trend component, so the forecast follows whatever direction the series is currently moving in instead of assuming a fixed growth rate, consistent with the wave-like, non-constant growth pattern found earlier in this report. Second, it uses a weekly seasonal component (`seasonal_periods=7`), since COVID-19 reporting in this dataset shows a recurring day-of-week pattern rather than a smooth daily trend, and a model without a seasonal term would otherwise treat that recurring pattern as noise.

The forecast is fit fresh on every request instead of being precomputed and stored, in line with the on-the-fly processing already established for the rolling average in Task 4: the model reflects whatever data is currently in Snowflake at request time, not a snapshot taken earlier. In the dashboard, the forecast is drawn as a dashed continuation of the same chart used for the rolling average, so the switch from historical data to projection is visible right there instead of shown as a separate, disconnected chart.

6.3 Bonus: Clustering

The `/clusters` endpoint groups countries with KMeans based on three per-capita metrics: total confirmed cases per 100,000 population, peak daily new cases per 100,000 population, and case fatality rate. Per-capita figures were used instead of raw totals because raw case counts are dominated by population size. Ranking by total cases alone would mostly just reproduce a ranking of countries by population, without saying much about how the pandemic affected each country's population differently. Population figures came from the same Snowflake Public Data (Free) listing used for enrichment in Task 2, joined by ISO alpha-2 code for the same reason given there: it is a reliable join key where country names are not.

Because the three features sit on very different scales (per-100k case counts can run into the tens of thousands, while a case fatality rate is a small percentage), each feature is standardized (zero mean, unit variance) before clustering. Skipping this step would leave KMeans effectively clustering on case counts alone, since the algorithm groups points by numerical distance, and a feature with a much larger numerical range dominates that distance calculation no matter how meaningful the other features are. A small number of countries with missing values in one of the three engineered metrics (no recorded case count on any date, which happens for a handful of very small territories in the dataset) are excluded from clustering instead of letting a missing value propagate into the result.

In the dashboard, the resulting clusters appear as a scatter plot of total cases per 100k against case fatality rate, colored by cluster and labeled by country. A cluster can be read straight off its position on the chart. A cluster sitting in the lower-right represents countries with widespread transmission but comparatively low fatality, while one in the upper-left represents the opposite: limited spread but disproportionately high fatality, which might point to differences in testing capacity or healthcare access rather than the virus behaving differently between those countries.

7. TASK 7 – PERFORMACE OPTIMIZATION

7.1 Constraint Specific to Dataset

Most Snowflake performance features (a clustering key, the search optimization service, a materialized view, a Dynamic Table) require the account to own the table being optimized. `JHU_COVID_19` is not owned by this account. It is a Secure Data Share, as established earlier, which means this account only has read access to it. This was confirmed directly: `SHOW TABLES` on `JHU_COVID_19` returns a blank owner column and no clustering

key, and an attempt to run `ALTER TABLE JHU_COVID_19 CLUSTER BY (COUNTRY_REGION, DATE)` was rejected. A second attempt, creating a Dynamic Table directly inside the shared database, failed with a clearer, Snowflake-specific message: "Creating dynamic_table on shared database 'COVID19_EPIDEMIOLOGICAL_DATA' is not allowed." Both confirm the same constraint from two different angles before any workaround was attempted, rather than assuming it and moving on.

7.2 Bottleneck

The constraint above does not mean the underlying queries cannot be optimized. It means the optimization cannot happen by modifying the shared table itself. Looking at where the real cost sits, the repeated aggregation pattern established earlier turns out to be the bottleneck: `/cases`, `/cases/rolling-average`, `/cases/forecast`, and `/clusters` all aggregate the same 9,738,292-row table with `SUM(CASES)` across subregions, on every single request, to correct for the county-level reporting granularity found in Task 2. That aggregation is identical every time it runs for a given country. Recomputing it from scratch per request, rather than once, is the actual performance problem, not the shared table's lack of a clustering key by itself.

7.3 Solution: An Owned, Pre-Aggregated Dynamic Table

Since the shared table cannot be altered, a new database, `COVID19_ANALYTICS`, was created to hold objects this account does own. Inside it, a Dynamic Table named `NATIONAL_DAILY_CASES` was defined, reading from `JHU_COVID_19` and pre-computing the exact aggregation described earlier: one row per country, case type, and date, with `SUM(CASES)` and `SUM(DIFFERENCE)` already applied across subregions. A Dynamic Table was chosen over a plain table populated by a script for a specific reason: it refreshes itself on a schedule (set to a 1-hour target lag here) without a separate ETL job to maintain, which matters given that the source data updates on the provider's side outside of this account's control.

Building this table surfaced a real trade-off during testing. The first version used `ANY_VALUE(ISO3166_1)` to pick a representative ISO code per country, and Snowflake responded by falling back to a full refresh instead of an incremental one, because `ANY_VALUE` is non-deterministic and Dynamic Tables cannot track incremental changes through a non-deterministic function. Replacing it with `MAX(ISO3166_1)`, a deterministic aggregate that returns the same result regardless of row order, fixed this. The table now refreshes incrementally, confirmed by `SHOW DYNAMIC TABLES` reporting `refresh_mode = INCREMENTAL`.

Because COVID19_ANALYTICS.PUBLIC.NATIONAL_DAILY_CASES is owned by this account, a clustering key could be added to it, unlike the source table. It was clustered by (CASE_TYPE, COUNTRY_REGION, DATE), matching the exact filter and sort pattern every API endpoint already uses (WHERE CASE_TYPE = ... AND COUNTRY_REGION = ... ORDER BY DATE), so Snowflake can prune micro-partitions down to the small slice relevant to a single country instead of scanning the whole table.

This surfaced a second real gotcha during re-testing: re-running CREATE OR REPLACE DYNAMIC TABLE on its own drops the clustering key that a previous ALTER TABLE ... CLUSTER BY had set. SHOW DYNAMIC TABLES confirmed an empty cluster_by column right after a re-run of the CREATE OR REPLACE statement, even though the same table had been clustered correctly before that. The fix is procedural rather than a code change: the ALTER TABLE ... CLUSTER BY statement has to be re-run immediately after the CREATE OR REPLACE statement every time the table is recreated, not just once at initial setup. task7_performance_optimization.sql documents this explicitly so it is not missed on a future re-run.

The four Snowflake-backed endpoints in api/main.py (/countries, /cases, /cases/rolling-average indirectly through /cases, and /clusters) were updated to read from this table instead of aggregating JHU_COVID_19 directly. The /clusters endpoint in particular went from a query with two levels of nested aggregation over the raw table to a query that just takes MAX() of already-aggregated rows, since the heavy aggregation now happens once, inside the Dynamic Table's own refresh, rather than inside every API request that needs it.

7.4 Measured Result

The same query (national daily case totals for the United States, confirmed case type) was run twice, once against the original aggregation over JHU_COVID_19 and once against NATIONAL_DAILY_CASES, and the two runs were compared using INFORMATION_SCHEMA.QUERY_HISTORY:

	Bytes Scanned	Elapsed Time
Before (aggregate <i>JHU_COVID_19</i> per request)	267,457,024	249 ms
After (read from <i>NATIONAL_DAILY_CASES</i>)	17,546,752	165 ms

That is roughly 15 times less data scanned and about a third less elapsed time, on a warehouse that was already idle and running at X-Small, the smallest available size. The row count behind this drop went from 9,738,292 (one row per subregion per case type per date) to 604,123 (one row per country per case type per date) after the Dynamic Table's aggregation,

about 16 times fewer rows to scan for the same query. The gap between the two ratios (16x fewer rows but 15x less data scanned) is expected, since NATIONAL_DAILY_CASES has fewer columns than JHU_COVID_19 and clustering additionally lets Snowflake skip micro-partitions outside the requested country before even reading them.

This result matters beyond a single query. Every endpoint that used to aggregate the raw table now benefits from the same reduction, and the saving scales with usage: a query run once a day saves a small amount of compute, while the same query run repeatedly through the dashboard, at every page load and every country switch, saves proportionally more, since the fix removes a fixed cost from every single one of those requests rather than from one query in isolation.

8. TASK 8 – CACHING

8.1 What Gets Cached and Why

Six endpoints were identified as good caching candidates: /countries, /cases, /cases/rolling-average, /vaccinations, /cases/forecast, and /clusters. All six either query Snowflake or, in the case of the last two, fit a model on every call (Holt-Winters for forecasting, KMeans for clustering), which is the most expensive work the API does per request. The two MongoDB-backed endpoints, /annotations and /external-sources, were deliberately left uncached. An annotation added through POST /annotations is expected to appear immediately in the dashboard's list on the next GET /annotations call, since that refresh-after-submit behavior was already built in Task 5. Caching that read would delay a write from showing up, for no real performance benefit, since a MongoDB point read against a small collection is already fast.

8.2 Design: A Shared In-Memory Cache, Not an External Service

Caching is implemented with *cachetools.TTLCache* in a new module, *api/cache.py*, rather than an external cache such as Redis. The reasoning follows from how this API actually runs: it is a single process, and every cached endpoint's underlying data changes on a schedule this project already controls, namely *COVID19_ANALYTICS.PUBLIC.NATIONAL_DAILY_CASES*'s 1-hour *TARGET_LAG* from Task 7. An in-memory cache with a matching 1-hour expiry keeps cached results from going stale any longer than the source data itself would allow, without introducing a second service (a Redis instance) that would need its own configuration, connection handling, and failure modes, for a project whose API runs as one process on one machine.

A single *@cached* decorator wraps each of the six endpoint functions. It builds a cache key from the function's fully qualified name plus its call arguments, so */cases?country=Germany* and */cases?country=France* are cached as separate entries rather than colliding. Because the decorator is applied directly to the underlying Python function, not to some separate HTTP-layer wrapper, endpoints that call another cached endpoint internally also benefit: */cases/forecast* calls *get_cases()* directly rather than making an HTTP request back to */cases*, and that internal call still hits the same cache */cases* itself uses.

8.3 Verifying Cache Actually Works

Two diagnostic endpoints were added specifically to make caching observable rather than something that has to be taken on faith: *GET /cache/stats* returns hit and miss counts plus the cache's current size, and *POST /cache/clear* empties it on demand. This was verified directly through the running API rather than assumed from the code alone. *GET /cache/stats* was called first and returned a clean baseline (*hits: 0, misses: 0, current_size: 0*). *GET /cases?country=Germany&case_type=Confirmed* was then called once, which correctly queried Snowflake and returned data. The identical request was called a second time with the exact same parameters, and *GET /cache/stats* was checked again afterward: it returned *hits: 1, misses: 1, current_size: 1*, matching the expected outcome exactly. One miss for the first call that had to query Snowflake, one hit for the second call that was served from memory instead, and one entry sitting in the cache for that specific country/case-type combination.

9. TASK 9 – PATTERN RECOGNITION

9.1 What Pattern to Look For

Earlier analysis already noted, from a rolling-average chart, that the pandemic unfolded in discrete waves rather than growing at a steady rate, and named the Omicron surge around January 2022 as a specific example. Task 9 turns that visual, manual observation into a formal, repeatable definition: a wave is a sustained rise in the 7-day rolling average of new cases, followed by a sustained fall. *MATCH_RECOGNIZE*, Snowflake's row pattern matching clause, was used to detect this shape directly in SQL, across every country in the dataset, rather than continuing to spot waves by eye one chart at a time.

9.2 Constraint With PREV()

Snowflake's own documentation for *MATCH_RECOGNIZE* defines a rise/fall ("V-shape") pattern using *PREV()* inside the *DEFINE* clause, comparing each row to the one immediately before it. That exact documented example was tried first, against a trivial five-row table with no window functions involved at all, to rule out anything specific to this project's data. It failed with: "Unsupported feature 'this window function and semantic combination does currently not support pattern variable predicates.'" Since this reproduced on a minimal case straight from Snowflake's own docs, it points to an account or account-edition-level restriction on using *PREV()* inside *DEFINE*, not a mistake in how it was written here.

The workaround: the "is this row higher or lower than a previous row" comparison was computed as a plain boolean column beforehand, using *LAG()*, a normal window function that carries no such restriction, and *DEFINE* was written to reference that already-computed column directly. *MATCH_RECOGNIZE* itself does not need to evaluate any navigation function once the comparison is already sitting in the table as *TRUE/FALSE*.

9.3 Single-Day Noise Breaks Long Trends

The first working version compared each row only to the day immediately before it (*LAG(..., 1)*). Testing it against the United States confirmed case series showed it missed the entire December 2021-January 2022 Omicron surge, the single largest wave in the whole dataset, entirely. A single quieter reporting day on 2021-12-13, in the middle of an otherwise clear multi-week rise, was enough to break the strict day-over-day "is this row higher than the previous one" definition into two fragments, neither of which was long enough on its own to

satisfy the pattern's minimum run length. Comparing each row against the value three days earlier instead of one day earlier ($LAG(..., 3)$) tolerates that kind of single-day noise without needing a longer smoothing window, and the Omicron wave was then detected correctly as a single, continuous match.

9.4 Result

Run against the United States, the query detects 30 waves across the full 2020-01-22 to 2023-03-09 date range. The largest by peak smoothed value is:

Wave Start	Peak Date	Wave End	Peak Smoothed New Cases
2021-12-15	2022-01-16	2022-01-19	806,966

This is the same Omicron wave already identified three separate times earlier in this project, through three different methods: visually, in the 7-day rolling average chart built in Task 5; as a specific dated entry in the MongoDB annotations collection from Task 3 ("Omicron wave peak in the US," dated 2022-01-10); and now a third time, independently and algorithmically, through MATCH_RECOGNIZE, landing on a peak date within days of the other two. Three unrelated methods converging on the same event is stronger evidence for a finding than any single one of them would be alone.

Running the identical query for Germany instead of the United States surfaces that country's two largest waves in the January-April 2022 window, consistent with Germany's own Omicron/BA.2 wave during that period. That the same pattern definition, with no country-specific tuning, correctly locates each country's major wave in the period it actually occurred, confirms this is a genuine property of the underlying data rather than an artifact of how one country happens to report figures.

10. TASK 10 – CODE AND REPOSITORY

All code and configuration for this project, the API, the dashboard, the SQL scripts for every task, the NoSQL schema and setup script, the automated enrichment script, requirements.txt, and this report, is hosted in a public GitHub repository rather than included as appendices, since the assignment allows either option and a repository lets every file be browsed, cloned, and run directly, in its actual current state, rather than as a static copy pasted into a document.

Repository: <https://github.com/DeimantasBaltrenas/covid19-project>

The credential-handling convention established in Tasks 2 through 4, reading SNOWFLAKE_USER, SNOWFLAKE_PASSWORD, SNOWFLAKE_ACCOUNT,

and MONGODB_URI from a local .env file excluded from version control via .gitignore, applies to this repository as it does to every other part of the project. Cloning the repository and following the setup steps in README.md does not expose or require access to this account's own credentials; an evaluator or supervisor running the project needs only their own Snowflake trial account and MongoDB Atlas cluster, set up the same way Task 1 and Task 3 describe.

CONCLUSION AND INSIGHTS SUMMARY

The tasks above describe how the platform was built. This section pulls together what the platform showed about the pandemic itself, drawn from the analysis in Tasks 2, 5, 6, and 9.

The pandemic moved in distinct waves, not a steady climb. The right-skewed distribution of daily case counts found earlier in this report, and the MATCH_RECOGNIZE wave detection in Task 9, both point to the same conclusion from two different angles: case growth was concentrated into a small number of sharp surges rather than spread evenly across the three-year period. For the United States, 30 such waves were detected between January 2020 and March 2023, rather than one long, gradually changing trend.

The Omicron wave was, by a wide margin, the largest single event in the dataset. Its peak smoothed daily new-case count, about 807,000 for the United States around 16-19 January 2022, was roughly 3.2 times higher than the next-largest wave detected (about 251,000, in early 2021). This is not a conclusion drawn from a single method: it showed up independently in a manually built rolling-average chart, in a specific dated observation recorded in the MongoDB annotations collection during Task 3, and again in the algorithmic wave detection in Task 9, with all three landing on essentially the same date. The same pattern definition, applied to Germany without any country-specific adjustment, located that country's own largest waves in the same January-April 2022 window, which points to Omicron's spread being an international event on a comparable timeline, not a US-specific artifact of how one country counts or reports cases.

Missing vaccination data is a timeline artifact, not a data quality problem. Earlier analysis traced the vaccination coverage field's missingness directly to OWID_VACCINATIONS only holding records from the point vaccines became available, well after the case dataset's January 2020 start. Any analysis joining the two has to account for this gap being structural, tied to when the vaccination rollout actually began, rather than a sign of unreliable data.

Negative daily case counts reflect real-world reporting corrections, not errors. Earlier analysis confirmed that jurisdictions retroactively revise previously published totals, which shows up as a negative day-over-day change on the date the correction is published rather than on the date the cases were originally miscounted. Treating every negative value as invalid data to discard, an intuitive first reaction, would have removed a legitimate and necessary part of the running cumulative total.

Case intensity and case fatality did not move together across countries. The clustering built in Task 6 groups countries by three per-capita measures rather than raw totals,

specifically so that population size does not dominate the comparison, and the resulting clusters separate countries with high transmission but comparatively low fatality from countries with the opposite profile: more limited spread but a disproportionately high fatality rate. Since the underlying virus was the same in both groups, that separation points toward differences in testing capacity, healthcare access, or population age structure between countries as a more likely explanation than the virus behaving differently from one country to the next, though this project's data does not include the healthcare-system variables needed to confirm that explanation directly.

Reporting granularity is not a fixed property of a country. Germany's switch from national-level to per-Bundesland reporting partway through 2020, established earlier in this report, is itself a finding about how the pandemic was tracked, not just a data engineering obstacle: the level of detail available for a given country can change mid-pandemic, which means any cross-country or cross-time comparison needs to check reporting structure across the full date range being compared, not assume it stayed constant just because it did in whichever period was checked first.